

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

df = pd.read_csv("binary.csv")

```

```
df
```

	admit	gre	gpa	rank
0	0	380	3.61	3
1	1	660	3.67	3
2	1	800	4.00	1
3	1	640	3.19	4
4	0	520	2.93	4
..	...	...	...	...
395	0	620	4.00	2
396	0	560	3.04	3
397	0	460	2.63	2
398	0	700	3.65	2
399	0	600	3.89	3

```
[400 rows x 4 columns]
```

```
df.head()
```

	admit	gre	gpa	rank
0	0	380	3.61	3
1	1	660	3.67	3
2	1	800	4.00	1
3	1	640	3.19	4
4	0	520	2.93	4

```
df.tail()
```

	admit	gre	gpa	rank
395	0	620	4.00	2
396	0	560	3.04	3
397	0	460	2.63	2
398	0	700	3.65	2
399	0	600	3.89	3

```
df.shape
```

```
(400, 4)
```

```
df.info()
```

<class 'pandas.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	admit	400 non-null	int64
1	gre	400 non-null	int64
2	gpa	400 non-null	float64
3	rank	400 non-null	int64

dtypes: float64(1), int64(3)
memory usage: 12.6 KB

```
df.isnull().sum()
```

```
admit    0
gre      0
gpa      0
rank     0
dtype: int64
```

```
df.describe()
```

	admit	gre	gpa	rank
count	400.000000	400.000000	400.000000	400.000000
mean	0.317500	587.700000	3.389900	2.485000
std	0.466087	115.516536	0.380567	0.944460
min	0.000000	220.000000	2.260000	1.000000
25%	0.000000	520.000000	3.130000	2.000000
50%	0.000000	580.000000	3.395000	2.000000
75%	1.000000	660.000000	3.670000	3.000000
max	1.000000	800.000000	4.000000	4.000000

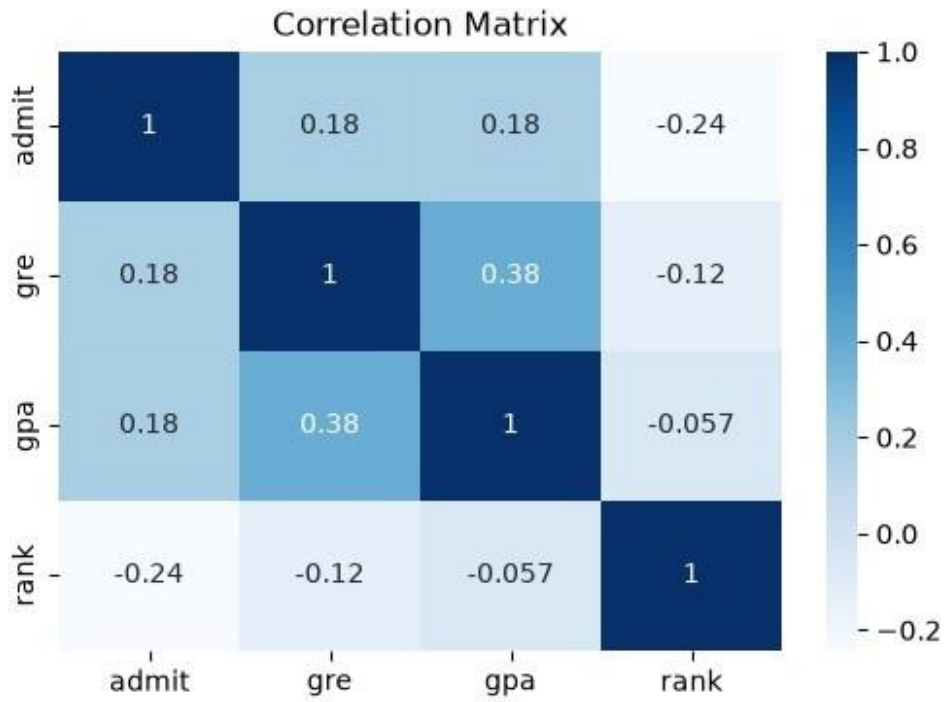
```
df.duplicated().sum()
```

```
np.int64(5)
```

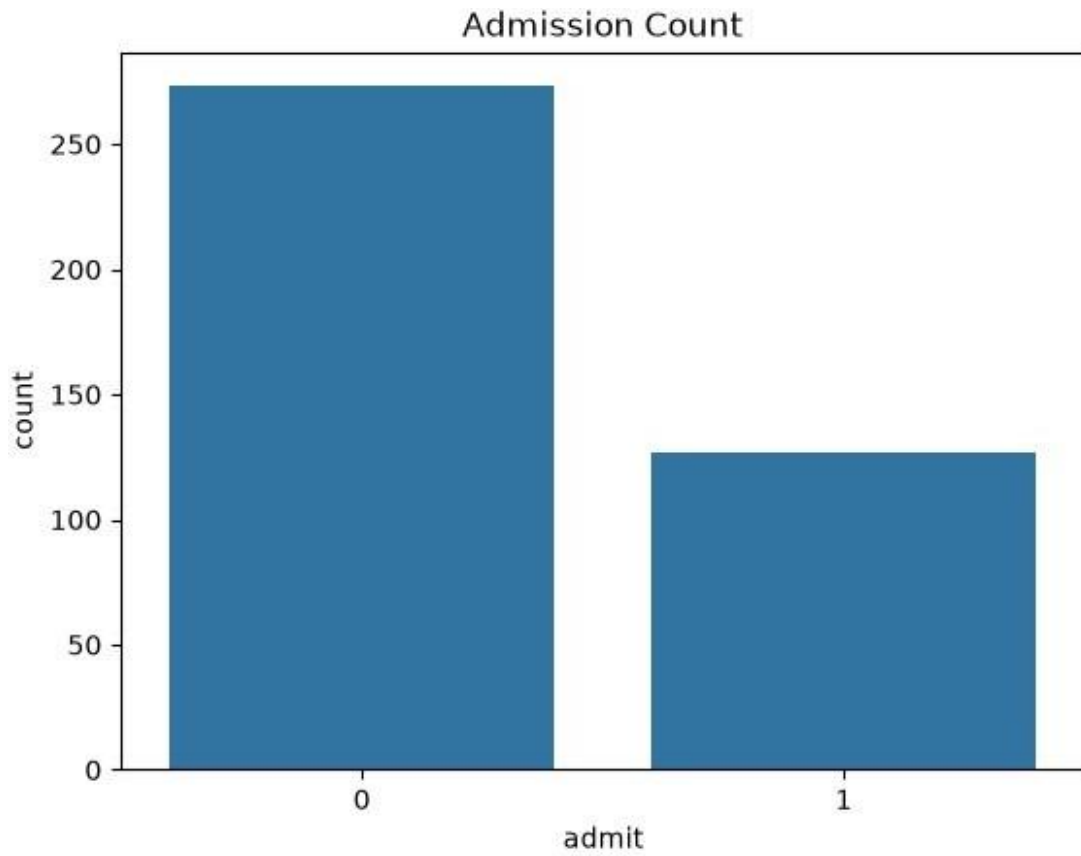
```
df.corr()
```

	admit	gre	gpa	rank
admit	1.000000	0.184434	0.178212	-0.242513
gre	0.184434	1.000000	0.384266	-0.123447
gpa	0.178212	0.384266	1.000000	-0.057461
rank	-0.242513	-0.123447	-0.057461	1.000000

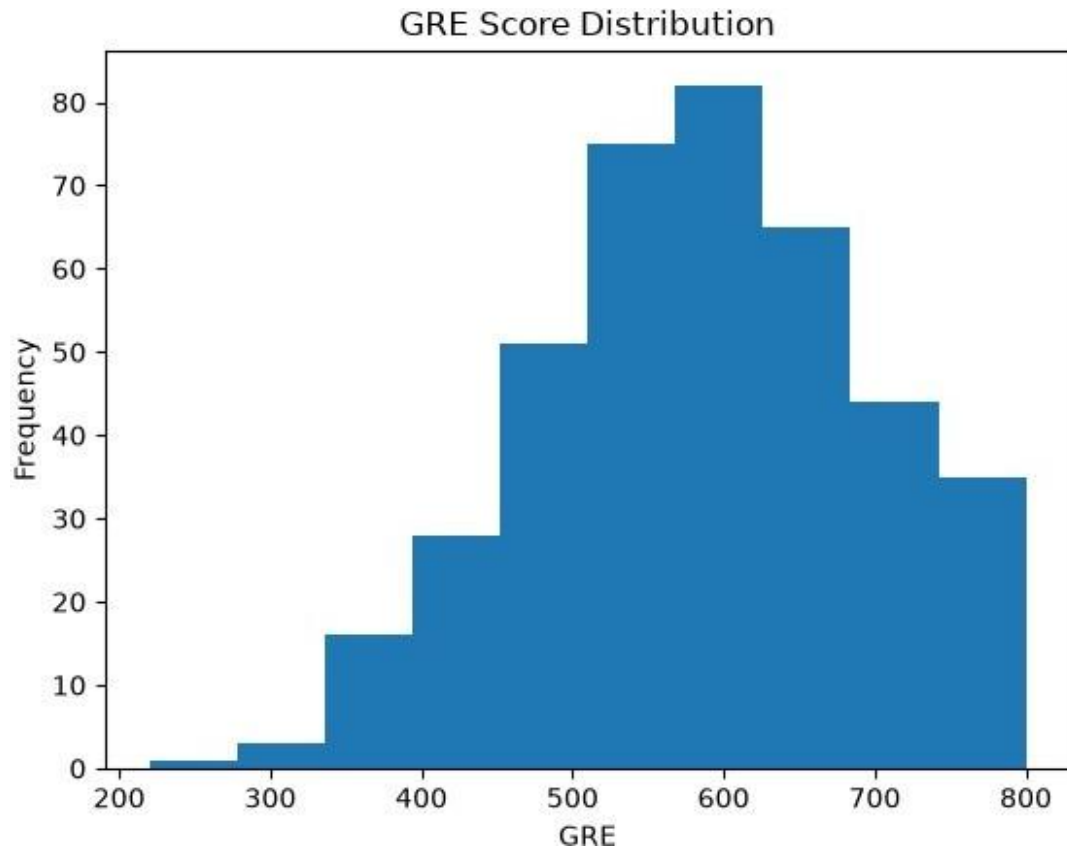
```
plt.figure(figsize=(6,4))
sns.heatmap(df.corr(),annot=True,cmap='Blues')
plt.title("Correlation Matrix")
plt.show()
```



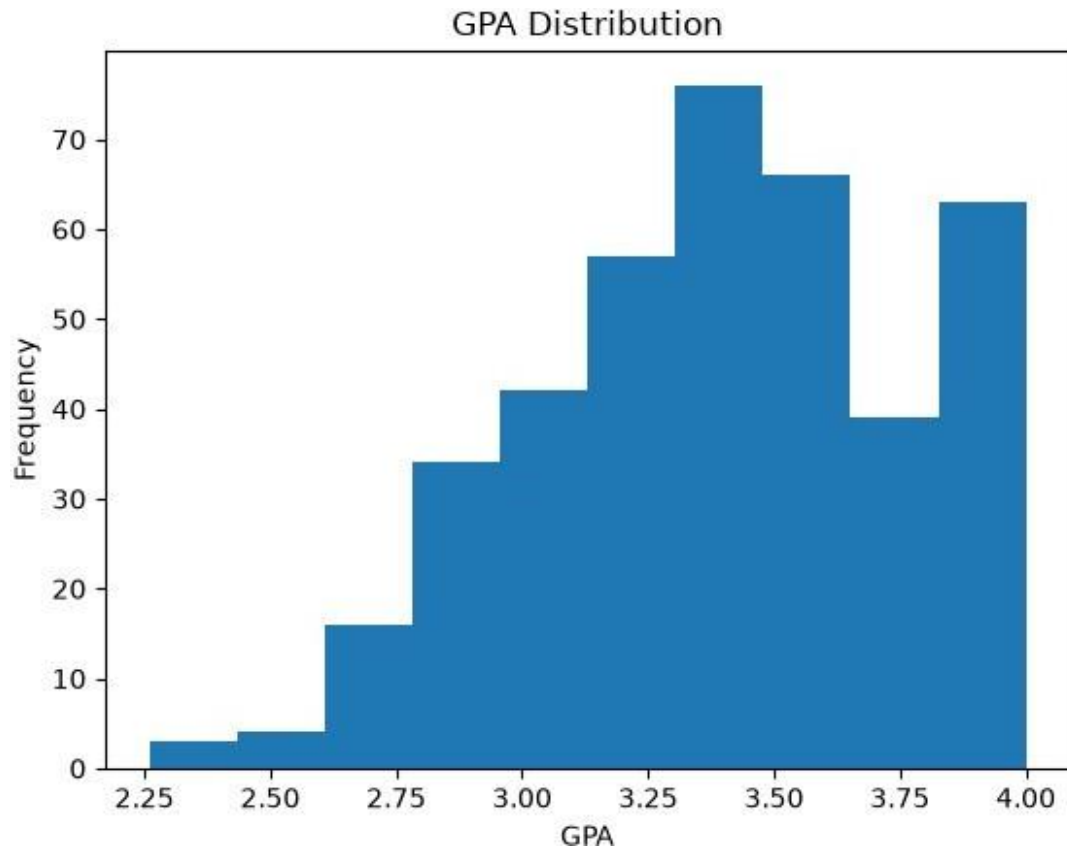
```
sns.countplot(x='admit',data=df)
plt.title("Admission Count")
plt.show()
```



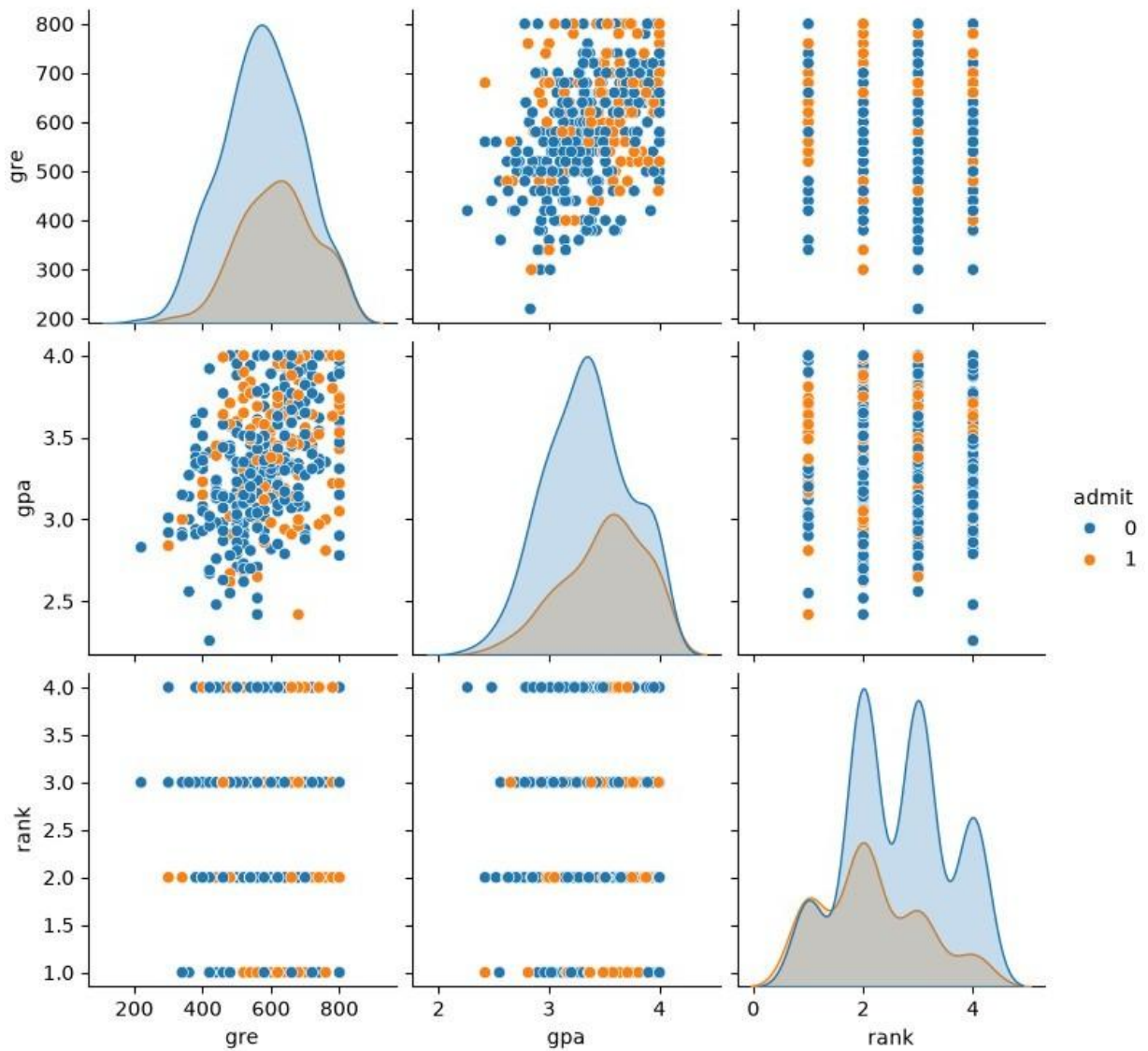
```
plt.hist(df['gre'],bins=10)
plt.title("GRE Score Distribution")
plt.xlabel("GRE")
plt.ylabel("Frequency")
plt.show()
```



```
plt.hist(df['gpa'],bins=10)
plt.title("GPA Distribution")
plt.xlabel("GPA")
plt.ylabel("Frequency")
plt.show()
```



```
sns.pairplot(df, hue='admit')  
plt.show()
```



```
X=df[['gre','gpa','rank']]
```

```
y=df['admit']
```

```
X.head()
```

	gre	gpa	rank
0	380	3.61	3
1	660	3.67	3
2	800	4.00	1
3	640	3.19	4
4	520	2.93	4

```
y.head()
```

```
0    0
1    1
2    1
3    1
4    0
Name: admit, dtype: int64

X_train,X_test,y_train,y_test=train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)

(320, 3)
(80, 3)
(320,)
(80,)

model=LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)

LogisticRegression(max_iter=1000)
y_pred=model.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)

print("Accuracy =",accuracy)

Accuracy = 0.6625

cm=confusion_matrix(y_test,y_pred)

print(cm)

[[51  2]
 [25  2]]

plt.figure(figsize=(5,4))

sns.heatmap(cm,
            annot=True,
```

```

        fmt='d',
        cmap='Greens')

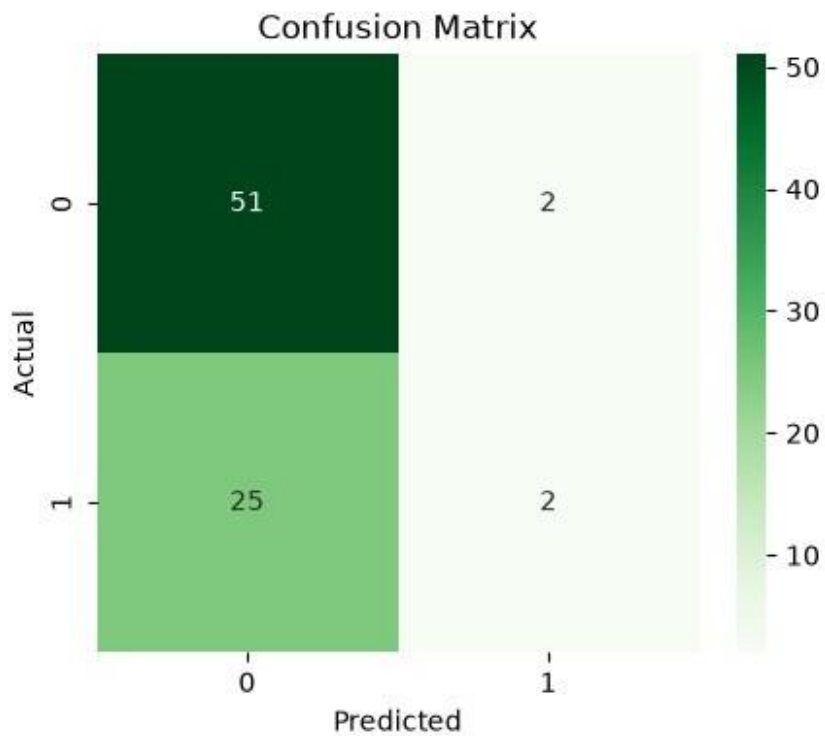
plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()

```



```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.67	0.96	0.79	53
1	0.50	0.07	0.13	27
accuracy			0.66	80
macro avg	0.59	0.52	0.46	80
weighted avg	0.61	0.66	0.57	80

```
print(model.coef_)
```

```
[[ 0.00185285  0.78297643 -0.60224243]]
```

```
print(model.intercept_)  
[-3.13756163]  
  
# New student data  
student = pd.DataFrame({  
    'gre': [650],  
    'gpa': [3.7],  
    'rank': [2]  
})  
  
prediction = model.predict(student)  
  
print("Prediction:", prediction)  
Prediction: [0]  
  
if prediction[0] == 1:  
    print("Student is likely to be admitted.")  
else:  
    print("Student is not likely to be admitted.")  
  
Student is not likely to be admitted.  
  
probability = model.predict_proba(student)  
  
print("Admission Probability:")  
print(probability)  
  
Admission Probability:  
[[0.55988162 0.44011838]]
```